

软件体系结构题库详解

该文档为 老师提供的题库 的答案，考试将从该题库中每种题型抽出一半题目进行考察，答案由AI给出

Prompt: 我一共发给你了15个pdf文档和一个md文档，其中有14个pdf文档为老师发布的课件，1个pdf为老师提供的期末复习大纲，md文档为老师提供的期末考试题库，声称期末考试将完全考试题库中的原题，但是我这门课完全没上过课没学过，涉及这门课的任何稍微专业一点的词我都完全不认识完全不懂，所以现在我需要你基于课件和复习提纲，来给题库中的所有题加上十分详细的讲解以便于我理解和记忆，只给答案的话显然我记不住，你需要产出一份包含原题和详解的文档，格式为（原题目、答案、详细讲解（显然你不止要讲解答案、还应该详细讲解题目、非答案的其他选项、以及此题整道题所涉及的内容的扩展）），每道题都应该遵循此结构，也就是说整体结构为 {（原题目、答案、详细讲解）（原题目、答案、详细讲解）...}

注意，你给出的答案应该看起来像答案，也就是应该符合考试的场景，比如简答题不需要太过详细，如果答案有多点，那就一点一句话；如果只有少数两点或三点或者无需分点作答，则就用两三段话或者一长段话。

题型	分值（分）	数量（道）	总计（分）
单选题	2	10	20
填空题	2	5	20
简答题	5	6	30
综合题	15	2	30

一、单选题

第1题

某电商系统采用 MVC 架构。在一次系统升级过程中，开发人员需要修改商品推荐算法，但要求不影响用户界面和请求处理逻辑。从 MVC 设计模式的思想来看，开发人员主要应修改（ ）。
A View层 B Controller层 C Model层 D View层和Controller层

答案：C

详细讲解：

MVC 是 Model（模型层）、view（视图层）、Controller（控制器层）三个单词的缩写，是一种典型的“关注点分离”设计模式，核心是让界面、业务逻辑、请求调度各司其职，修改其中一块尽量不影响其他部分。

- **View层（视图层）**：对应用户能直接看到、操作的界面，比如电商的商品展示页、按钮、排版，只负责展示数据和接收用户输入，对应题干里的“用户界面”。因此A、D选项不符合要求。
- **Controller层（控制层）**：负责请求的转发和调度，相当于“前台接待”。比如用户点击“看推荐商品”，控制器收到请求后，知道要去调用推荐算法，但它本身不实现算法，只做中转，对应题干里的“请求处理逻辑”。因此B选项不符合要求。
- **Model层（模型层）**：负责核心业务逻辑和数据存储，是系统的“大脑”，比如价格计算、推荐算法、数据读写都属于这一层。商品推荐算法是典型的业务逻辑，完全封装在Model层中，修改算法只需要调整Model层，不会触动界面和调度逻辑，完全符合题干要求。

扩展知识点： MVC 的核心价值是解耦，方便团队分工（前端开发View，后端开发Model和Controller），也提升了可维护性——改页面不用动业务逻辑，改算法不用动界面。

第2题

选项中不属于“4+1”视图模型的是：（ ）。

A 逻辑视图 B 物理视图 C 连接视图 D 开发视图

答案：C

详细讲解：

“4+1”视图模型是架构师 Kruchten 提出的经典架构描述方法，核心思想是：单一视角无法完整描述复杂系统，需要从5个不同维度刻画架构，就像描述房子需要户型图、水电图、装修图、位置图，再加一个生活场景图。

其中“4”是4个核心视图，“1”是串联所有视图的场景视图：

1. **逻辑视图**：面向最终用户，描述系统的功能模块划分，比如电商的用户模块、商品模块，对应功能需求。
2. **开发视图**：也叫模块视图，面向开发人员，描述代码的组织方式、包结构和依赖关系。
3. **进程视图**：面向系统集成人员，描述系统运行时的进程、线程、并发与通信，关注性能、吞吐量。
4. **物理视图**：面向系统工程师，描述软件如何部署到硬件上，比如服务器数量、网络拓扑。
5. **场景视图**：也叫用例视图，用具体的业务场景把4个视图串联起来，验证架构的合理性。

C选项“连接视图”是虚构的干扰项，不属于4+1视图模型。

扩展知识点：4个核心视图中，逻辑视图、开发视图描述系统的**静态结构**，进程视图、物理视图描述系统的**动态结构**。

第3题

关于 RAG (Retrieval-Augmented Generation) 技术，下列说法错误的是哪一项？

- A. RAG 通过检索外部知识增强大语言模型生成能力
- B. RAG 能够在不重新训练模型的情况下引入最新知识
- C. RAG 检索到的内容通常作为 Prompt 上下文输入给大语言模型
- D. RAG 与模型微调 (Fine-tuning) 本质相同

答案：D

详细讲解：

RAG 中文叫**检索增强生成**，可以理解为给大语言模型“外挂了一个知识库”，让模型可以“开卷考试”，答题前先查资料，再生成答案。

- A选项：RAG 的核心逻辑就是“检索外部知识 + 辅助生成”，通过召回相关的外部资料，提升大模型回答的准确性，减少幻觉，表述正确。
- B选项：RAG 不需要修改大模型本身，只要更新外部知识库就能引入新知识，比如公司更新了产品手册，直接把新手册存入知识库即可，不用花费大量成本重新训练模型，这是RAG的核心优势，表述正确。
- C选项：RAG 的标准流程是：用户提问 → 检索知识库找到相关内容 → 将“用户问题+检索到的资料”拼接成提示词 (Prompt) → 发给大模型生成答案，检索内容就是作为Prompt的上下文输入的，表述正确。
- D选项：RAG和微调本质完全不同。**微调 (Fine-tuning)** 是用新数据训练模型，修改模型内部的参数，让模型“学会”新知识，是改变模型本身；而RAG不碰模型参数，只是外挂知识库，相当于“考试查资料”，模型本身没记住知识。两者原理、成本、适用场景都不一样，表述错误。

扩展知识点：RAG适合知识更新频繁、要求答案可溯源的场景（比如企业客服、知识库问答）；微调适合让模型掌握固定风格、特定技能，适合知识不频繁变动的场景。

第4题

设计模式一般用来解决什么样的问题？

- A. 同一问题的不同表相
- B. 不同问题的同一表相
- C. 不同问题的不同表相
- D. 以上都不是

答案：A

详细讲解：

设计模式是软件开发领域的“解题模板”：开发者在长期实践中，把**同一类反复出现的核心问题**的通用、可复用的解决方案总结下来，就形成了设计模式。

比如“保证一个类只能有一个实例”是同一个核心问题，不管是日志管理器、数据库连接池还是配置管理器，都是这个问题的不同场景（不同表相），都可以用“单例模式”解决。

也就是说，设计模式针对的是同一个本质问题，适配这个问题在不同业务、不同场景下的不同表现形式，即“同一问题的不同表相”。

扩展知识点：经典的23种设计模式分为三大类：创建型模式（管对象怎么创建）、结构型模式（管类和对象怎么组合）、行为型模式（管对象之间怎么交互）。

第5题

某大型在线购物平台在体系结构设计过程中，架构师从不同角度对系统进行了描述。根据 Kruchten 提出的 4+1 视图模型，其中对“系统在多服务器环境下的并发处理能力和运行时进程之间的通信机制”的描述属于（ ）。

- A 逻辑视图 B 开发视图 C 进程视图 D 物理视图

答案：C

详细讲解：

本题考查4+1视图各自的关注点，抓住题干关键词“运行时”“并发处理”“进程通信”即可定位答案。

- A 逻辑视图：关注功能需求，即系统能给用户提供服务，比如下单、支付、退款功能，不涉及运行时的进程和并发，错误。
- B 开发视图：关注代码的静态组织方式，比如代码分包、模块依赖，是开发阶段的结构，不关心运行时的动态行为，错误。
- C 进程视图：关注系统运行时的动态特性，核心就是进程/线程的调度、并发处理、进程间通信，重点解决性能、吞吐量、容错等非功能需求，完全匹配题干描述，正确。
- D 物理视图：关注软件如何部署到硬件上，比如有几台服务器、分别部署什么服务、网络怎么连，是硬件部署视角，不关注进程内部的通信机制，错误。

扩展知识点：4+1视图对应不同的角色：逻辑视图给最终用户看，开发视图给程序员看，进程视图给系统集成人员看，物理视图给系统工程师看。

第6题

软件体系结构风格是指什么？

- A. 软件编码规范
- B. 描述特定应用领域系统组织方式的惯用模式
- C. UML 建模工具
- D. 数据库设计方法

答案：B

详细讲解：

软件体系结构风格（也叫架构风格），是长期实践中沉淀下来的、描述某一类系统组织方式的**惯用架构模板**。比如编译器常用管道-过滤器风格，企业系统常用分层风格，它们都是被反复验证、可以直接复用的架构范式。

- A 编码规范是写代码的格式要求（比如变量命名、缩进），属于最底层的代码细节，和高层的架构风格不是一个层级，错误。
- B 完全符合架构风格的定义：针对特定领域，描述系统组织方式的惯用、可复用模式，正确。
- C UML是用来画架构图的建模工具，工具本身不是架构风格，错误。
- D 数据库设计只是系统设计的一小部分，架构风格是整个系统的顶层组织方式，范畴远大于数据库设计，错误。

扩展知识点：经典的架构风格分为五大类：数据流风格、调用/返回风格、独立构件风格、虚拟机风格、数据为中心风格。

第7题

在三层 C/S 体系结构中，（ ）是最重要的构件。

- A 中间件 B 末尾件 C 功能层 D 数据层

答案：A

详细讲解：

三层C/S（客户端/服务器）架构将系统按职责拆分为三层：**表示层（界面交互）、功能层（业务逻辑）、数据层（数据存储）**。但从构件的角度，**中间件是最重要的构件**。

中间件是通过 API 定义的分布式软件层，位于客户机与服务器、服务器与服务器之间，承担层间通信、数据传输、事务管理、异构系统适配等核心支撑功能，是实现三层架构解耦、分布式协同运行的关键基础，连接表示层、功能层与数据层，保障整个系统的交互与扩展能力。

- B “末尾件”是虚构的干扰项，没有这个概念，错误。
- C 功能层是三层架构的逻辑分层之一，承载业务逻辑处理，但它是逻辑层次而非构件实体，不符合“构件”的提问范畴。
- D 数据层负责数据的存储、读写与管理，是系统的基础数据支撑，但并非三层架构中最核心的构件。

扩展知识点：三层C/S架构的优势是：修改业务逻辑不用更新客户端，也不用改动数据库，大幅提升了可维护性；同时功能层可以独立部署，支持更高的并发量。

第8题

以下关于 MCP 协议描述正确的是哪一项？

- A. MCP 只能在本地环境中使用，无法支持远程服务调用
- B. MCP 与 Function Calling 完全等价，只是命名方式不同
- C. MCP 采用 Client-Server 架构，在 Agent 与外部工具之间添加中间层，统一封装工具调用逻辑
- D. MCP 全称是模型上下文协议，由 Anthropic 公司继 A2A 协议后提出的新协议

答案：C

详细讲解：

MCP 全称 Model Context Protocol（模型上下文协议），是一个开放的通用协议，用来标准化大模型/智能体与外部工具的通信方式，相当于给工具调用定了统一的“接口标准”。

- A选项：MCP采用客户端-服务器架构，支持远程部署和调用，不是只能本地使用，错误。
- B选项：Function Calling（函数调用）是大模型直接调用开发者预定义的函数，是一对一、厂商定制的；而MCP是标准化通用协议，支持动态发现工具、跨模型复用，两者本质不同，错误。
- C选项：MCP确实是客户端-服务器架构：Agent端作为客户端，外部工具封装成MCP服务器，双方通过标准协议通信。它相当于在Agent和工具之间加了一层统一适配层，不用每个工具都单独对接大模型，正确。
- D选项：MCP由Anthropic提出，但A2A（Agent-to-Agent，智能体间通信协议）是后续由Google提出的，MCP并非“继A2A之后”提出，时间顺序错误，错误。

扩展知识点： MCP的核心价值是标准化——只要符合MCP标准的工具，任何支持MCP的大模型都能直接使用，不用重复开发适配代码，大幅提升了工具的复用性。

第9题

软件危机的原因不包括哪一项？

- A. 用户需求不明确
- B. 缺乏正确的理论指导
- C. 软件规模越来越大
- D. 软件复杂度越来越低

答案：D

详细讲解：

软件危机是指早期软件开发中普遍存在的开发成本失控、进度延期、质量差、难以维护等一系列严重问题。

导致软件危机的核心原因：

1. **用户需求不明确**：很多项目初期用户说不清需求，开发中途频繁变更，导致项目返工失控。
2. **缺乏正确的理论指导**：早期软件开发是“手工作坊”模式，没有成熟的软件工程方法和设计理论，全靠程序员个人经验。
3. **软件规模越来越大**：随着计算机发展，软件从简单小程序变成百万、千万行代码的大型系统，复杂度指数级上升，原有开发方式无法支撑。
4. D选项“软件复杂度越来越低”与事实完全相反：**软件复杂度是越来越高的**，复杂度升高正是软件危机的核心诱因之一。

扩展知识点： 解决软件危机的核心方案是**软件工程**——用工程化的方法管理软件开发过程，包含方法、工具、过程三要素。

第10题

下列属于数据流风格的是哪一项？

- A. 分层结构
- B. MVC
- C. 管道-过滤器
- D. C/S

答案：C

详细讲解：

数据流风格的核心特征是：系统运行由数据流动驱动，数据像水流一样从一个构件流向下一个构件，构件之间只有数据交互，没有直接的函数调用。

- A 分层结构属于**调用/返回风格**，上层主动调用下层的服务，是显式调用关系，不属于数据流风格。
- B MVC是混合了事件驱动和调用返回的设计模式，不属于数据流风格。
- C 管道-过滤器是数据流风格的典型代表：过滤器是处理数据的构件，管道是传输数据的连接件；数据从入口流入，依次经过多个过滤器处理，最后输出结果，完全符合数据流风格的特征。
- D C/S（客户端/服务器）属于**调用/返回风格**，客户端主动调用服务器的服务，是显式调用关系，不属于数据流风格。

扩展知识点：数据流风格分为两个子类：

1. **批处理**：数据整批传递，前一个步骤完全跑完，才能开始下一个步骤，比如月末批量生成报表。
2. **管道-过滤器**：数据流式增量处理，边输入边处理边输出，比如编译器、视频转码、日志实时分析。

第11题

通常,一个 Web 服务可以分为 4 个逻辑层, 分别为数据层 (Data Layer)、数据访问层 (Data Access Layer)、业务层 (Business Layer) 和监听者 (Listener)。离客户端最近的是监听者, 离客户端最远的是 ()。

- A 数据层 (Data Layer)
- B 数据访问层 (Data Access Layer)
- C 业务层 (Business Layer)
- D 监听者 (Listener)

答案：A

详细讲解：

Web服务的四层是典型的分层架构，请求从客户端发出，从上到下逐层传递，顺序为：

客户端 → 监听者 (Listener) → 业务层 (Business Layer) → 数据访问层 (Data Access Layer) → 数据层 (Data Layer)

- 监听者：最上层，直接接收客户端的请求（比如HTTP请求），是系统的入口，离客户端最近。
- 业务层：处理核心业务逻辑，比如校验规则、计算逻辑。
- 数据访问层：专门负责和数据库交互，执行读写数据的操作。
- 数据层：最底层，就是数据库本身，负责持久化存储数据，离客户端最远。

扩展知识点：分层架构遵循“单向依赖”原则：上层只能调用紧邻的下层，不能越级调用，这样可以降低耦合，修改底层实现不影响上层。

第12题

下列哪项属于软件体系结构文档化过程的主要输出结果？

- A. 体系结构需求规格说明
- B. 体系结构概要设计说明
- C. 体系结构详细设计说明
- D. 体系结构总体框架说明

答案：A

详细讲解：

软件体系结构文档化，是将架构设计的成果整理成正式、规范的交付文档，其核心输出是**体系结构需求规格说明**（也叫软件体系结构规格说明书）。这份文档完整定义了架构的需求、设计方案、约束、多视图描述、质量属性目标等全部内容，是后续开发、评审、架构演化的核心依据。

- B 概要设计说明、C 详细设计说明，都是更底层的软件设计阶段文档，不属于架构级文档化的主要输出。
- D 总体框架说明只是架构文档的一部分内容，不是完整的主要输出结果。

扩展知识点：架构文档是所有项目相关方（客户、开发、测试、运维）沟通的共同语言，文档的完整性和准确性，直接决定了架构设计的落地效果。

第13题

某游戏公司欲开发一个大型多人即时战略游戏，游戏设计的目标之一是能够支持玩家自行创建战役地图，定义游戏对象的行为和地图之间的关系。针对该目标，公司应该采用（ ）架构风格最为合适。

A 管道-过滤器 B 隐式调用 C 主程序-子程序 D 解释器

答案：D

详细讲解：

题干核心需求是：支持用户自定义规则、自定义行为，也就是系统要能解析并执行用户编写的自定义内容，这是**虚拟机风格中解释器架构**的典型场景。

- A 管道-过滤器适合流式数据处理（比如转码、日志清洗），不支持自定义规则逻辑，错误。
- B 隐式调用（事件驱动）适合模块解耦，但无法支持用户自定义复杂的行为逻辑，错误。
- C 主程序-子程序是固定流程的调用结构，所有逻辑都是写死在代码里的，用户无法自定义，错误。
- D 解释器风格属于虚拟机风格，核心是在系统内构建一个“虚拟机”，可以解释执行自定义的脚本/规则。玩家自定义的地图、对象行为，就相当于一段自定义的“小程序”，解释器可以解析并运行这些内容，完美匹配需求。很多游戏的MOD系统、自定义地图功能，底层都是解释器架构，正确。

扩展知识点：解释器风格的常见应用：Java虚拟机（JVM）、Python/JavaScript等脚本语言、工作流引擎、低代码平台的自定义规则引擎。

第14题

某软件系统运行过程中，各功能模块无需直接调用彼此，而是通过发布和订阅事件完成通信。当某个模块状态发生变化时，系统自动通知所有关注该事件的模块进行处理。该体系结构风格属于（ ）。

A 主程序-子程序风格 B 分层体系结构风格
C 事件系统风格 D 管道过滤器风格

答案：C

详细讲解：

题干关键词“发布事件、订阅事件、模块不直接调用、事件触发自动通知”，完全匹配**事件系统风格（也叫隐式调用风格）**的特征。

事件系统风格的核心逻辑：构件不直接调用对方的方法，而是发布“事件”；其他构件可以订阅自己关心的事件；当事件触发时，系统自动调用所有订阅者的处理逻辑。

- A 主程序-子程序是显式调用，主程序主动调用子程序，和“不直接调用”的特征矛盾，错误。
- B 分层架构是上层显式调用下层，也是主动调用关系，不是事件订阅，错误。
- D 管道过滤器是数据驱动，靠数据流的流动触发处理，不是事件发布订阅，错误。

扩展知识点：事件系统风格的优点是模块高度解耦、扩展性强，新增功能只要订阅对应事件即可，不用修改原有代码；缺点是调试难度高，无法直接预知一个事件会触发多少后续动作，也难以控制执行顺序。

第15题

以下关于 SAAM 和 ATAM 的说法正确的是哪一项？

- A. SAAM 和 ATAM 均采用场景驱动的评估方法
- B. SAAM 主要分析质量属性之间的权衡关系
- C. ATAM 只能评估性能属性
- D. SAAM 和 ATAM 只关注功能需求

答案：A

详细讲解：

SAAM（软件架构分析方法）和ATAM（架构权衡分析方法），是两种最经典的基于场景的软件体系结构评估方法。

- A选项：两者都是**场景驱动**的评估方法——不用空洞的“性能好”“易维护”描述质量，而是用具体的、可衡量的场景来评估架构是否达标，表述正确。
- B选项：分析质量属性之间的权衡关系（比如性能和安全性的取舍）是ATAM的核心能力，SAAM主要侧重评估可修改性，不侧重权衡分析，错误。
- C选项：ATAM可以评估多个质量属性，包括性能、安全性、可修改性、可用性等，并非只能评估性能，错误。
- D选项：两者的核心评估对象是**非功能质量属性**，不是功能需求，错误。

扩展知识点：SAAM是最早的架构评估方法，简单易用，侧重可修改性；ATAM是在SAAM基础上发展而来的更全面的方法，核心是分析多个质量属性之间的敏感点和权衡点。

第16题

下列说法错误的一项是：（ ）。

- A 逻辑视图主要支持系统的功能需求，即系统提供给最终用户的服务
- B 开发视图也称模块视图，主要侧重于软件模块的组织和管理
- C 进程视图侧重于系统的运行特性，主要关注一些功能性需求，例如系统的性能和可用性
- D 物理视图主要考虑如何把软件映射到硬件上，它不需要考虑到系统性能、规模、可靠性等

答案：D

详细讲解：

本题考查4+1视图的基础定义，逐个分析：

- A选项：逻辑视图面向最终用户，核心就是承载系统的功能需求，描述用户能用到的服务和功能模块，表述正确。
- B选项：开发视图也叫模块视图，面向开发人员，关注软件模块的划分、组织和依赖管理，表述正确。
- C选项：进程视图关注运行特性，核心是性能、可用性、吞吐量等**非功能需求**，选项中“功能性需求”的表述略有瑕疵，但相比D选项的明显错误，D的错误更绝对。
- D选项：物理视图的核心工作，就是根据系统的性能要求、规模大小、可靠性目标，来设计硬件拓扑和软件部署方案。比如要支持高并发就要多部署服务器，要高可靠就要做容灾备份，这些都是物理视图必须考虑的内容。“不需要考虑”的表述完全错误。

扩展知识点：物理视图的输出一般是系统部署图，标注清楚每个节点部署什么服务、节点之间的网络连接方式。

第17题

某系统采用不同的体系结构设计方案进行开发。经过分析发现，方案 A 比方案 B 具有更好的可扩展性和可维护性。由此说明软件体系结构（ ）。

- A 只影响系统功能
- B 只影响开发成本
- C 会制约系统质量属性
- D 与系统质量无关

答案：C

详细讲解：

软件体系结构是系统的“骨架”，它不决定系统“能做什么”（功能），但决定了系统“做得怎么样”——也就是系统的**质量属性**（可扩展性、可维护性、性能、安全性、可靠性等）。架构设计的好坏，直接决定了这些质量属性的上限。

- A选项：架构不决定系统功能，同一个功能可以用多种架构实现，错误。
- B选项：架构不仅影响开发成本，还影响维护成本、系统性能、可靠性等方方面面，错误。
- C选项：可扩展性、可维护性都是典型的软件质量属性，不同的架构设计会带来不同的质量表现，架构直接制约着系统的质量上限，表述正确。
- D选项：架构是决定系统质量的核心因素，错误。

扩展知识点：ISO 9126软件质量模型定义了6大质量属性：功能性、可靠性、易用性、效率、可维护性、可移植性。

第18题

不属于软件体系结构核心模型最基本元素的是哪一项？

- A. 构件
- B. 连接件
- C. 配置
- D. 角色

答案：D

详细讲解：

软件体系结构核心模型有一个经典公式： $\text{软件体系结构} = \text{构件} + \text{连接件} + \text{约束（配置）}$ 。其中**构件、连接件、配置**是最基本的三大元素：

1. **构件**：系统中的功能单元，相当于“零件”，比如用户模块、数据库、过滤器，是具备独立功能的可重用单元。
2. **连接件**：构件之间的交互方式，相当于“连接线”，比如过程调用、管道、消息队列、事件总线。
3. **配置**：也叫拓扑，指构件和连接件的组合结构、约束规则，也就是零件的组装方式和整体布局。

“角色”是更细分的元素（和端口对应，是连接件的接口），不属于最基本的三大核心元素。

扩展知识点：更完整的核心模型还包含端口（构件的接口）和角色（连接件的接口），但最基础、最核心的是构件、连接件、配置三者。

第19题

以下关于软件体系结构描述方法说法错误的是（ ）。

- A 图形表达工具在软件设计中占据主导地位
- B 由于软件设计语言和模块内连接语言具有严格的语义基础，因此它们能支持较小的软件单元进行描述
- C 基于构件的系统描述语言将软件系统描述成一种是由许多以特定形式相互作用的特殊软件实体构造组成的组织或系统
- D 软件体系结构描述语言是参照传统程序设计语言的设计语言的设计和开发经验，重新设计、开发和使用时针对软件体系结构描述语言

答案：A

详细讲解：

本题考查体系结构的五类描述方法的特点。

- A选项：非标准的图形表达工具（矩形框+线条的草图）虽然直观，但语义不精确、不规范，只能用于草图绘制和商业展示，在专业的软件体系结构设计中并不占据主导地位。专业的架构描述会使用UML、体系结构描述语言（ADL）等语义更严谨的方法，表述错误。
- B选项：模块内连接语言（MIL）有严格的语义基础，适合描述小粒度软件单元的连接关系，表述正确。
- C选项：基于构件的描述语言，核心就是将系统描述为多个相互作用的构件实体的组合，表述正确。
- D选项：体系结构描述语言（ADL）借鉴了传统程序设计语言的设计经验，针对架构级的抽象特点重新设计，专门用于描述软件体系结构，表述正确。

扩展知识点：五类体系结构描述方法分别是：非标准图形符号、统一建模语言UML、形式化方法、模块内连接语言（MIL）、体系结构描述语言（ADL）。

第20题

下列哪项属于软件体系结构文档化过程的主要输出结果（ ）。

- A 体系结构需求规格说明
- B 体系结构概要设计说明
- C 体系结构详细设计说明
- D 体系结构总体框架说明

答案：A

详细讲解：

本题与第12题考点重复。软件体系结构文档化的核心、主要输出成果，是**体系结构需求规格说明**。它是架构设计阶段的正式交付物，完整覆盖架构需求、设计方案、视图、约束、质量目标等全部内容，是后续开发、评审、演化的核心依据。

概要设计、详细设计都属于更底层的设计阶段文档，不属于架构文档化的主要输出；总体框架说明只是架构文档的局部内容，不是完整的主要输出。

扩展知识点：基于体系结构的软件开发模型（ABSDM）中，文档化是独立的核心阶段，承接架构设计，指导后续实现。

二、填空题

第1题

MVC 是____、____和____三个单词的缩写。

答案：Model（模型）、View（视图）、Controller（控制器）

详细讲解：

MVC是经典的表现层设计模式，三个单词对应三个职责独立的组件：

1. **Model（模型层）**：封装核心数据和业务逻辑，是系统的功能核心，不依赖界面，独立于视图和控制器。
2. **View（视图层）**：负责数据的可视化展示，以及接收用户的输入操作，是用户直接交互的界面层。
3. **Controller（控制器层）**：作为中介，接收用户请求，调用对应的模型处理业务，再选择合适的视图把结果展示给用户。

MVC的核心设计思想是**关注点分离**，通过拆分职责降低耦合，提升代码的可维护性和可复用性。

第2题

在典型 RAG 系统中，首先需要对原始文档进行____处理，并通过____模型将其转换为向量存储到向量数据库中；用户提问时，系统检索相关内容并将其作为____输入大语言模型生成答案。

答案：分块（Chunking）、嵌入（Embedding）、上下文（Prompt）

详细讲解：

RAG（检索增强生成）的完整流程分为两个阶段：

阶段1：知识库构建

1. **分块处理**：原始文档（PDF、Word等）通常篇幅很长，无法直接向量化，需要先按语义拆分成大小合适的文本块，保证每块语义完整，同时适配模型的上下文长度。
2. **嵌入转换**：通过嵌入模型（Embedding模型），将每个文本块转换成数值向量。向量可以表征文本的语义：语义越相近的文本，向量在空间中的距离越近。
3. 存储：将向量和对应的原文存入向量数据库。

阶段2：问答生成

1. 用户提问，同样将问题转换为向量。
 2. 检索：在向量数据库中，通过相似度计算，召回和问题最相关的文本块。
 3. 生成：将检索到的文本作为**上下文**，和用户问题拼接成完整的提示词（Prompt），输入大语言模型，让模型基于上下文生成准确答案。
-

第3题

ABSD 的全称是：_____。

答案：Architecture-Based Software Design（基于软件体系结构的设计方法）

详细讲解：

ABSD是一种以架构为核心的软件开发方法，中文为**基于体系结构的软件开发方法**。

它的核心思想是：将软件体系结构作为开发的核心和基础，先完成系统的架构设计，再基于架构逐步细化、实现整个系统，整个开发过程围绕架构展开。

对应的完整开发模型叫 ABSDM（Architecture-Based Software Development Model，基于体系结构的软件开发模型），全过程分为6个阶段：体系结构需求、设计、文档化、复审、实现、演化。

第4题

三层 C/S 结构风格由_____、_____和_____构成。

答案：表示层、功能层、数据层

详细讲解：

三层C/S架构是在两层C/S基础上发展而来的分层架构，按职责将系统拆分为独立的三层：

1. **表示层：**最上层，直接面向用户，负责展示界面、接收用户输入，也就是客户端部分。
 2. **功能层：**也叫业务逻辑层、中间层，是三层的核心，封装所有业务规则和计算逻辑，承上启下，接收表示层的请求，处理业务后调用数据层。
 3. **数据层：**最底层，负责数据的持久化存储和读写，一般为数据库系统。
三层架构通过职责分离，实现了“高内聚、低耦合”，提升了系统的可维护性、可扩展性，也支持分布式部署。
-

第5题

体系结构评估中，一般采用_____、_____和_____三方面来对场景进行描述。

答案：刺激、环境、响应

详细讲解：

在SAAM、ATAM等基于场景的架构评估方法中，一个质量属性场景通过“刺激-环境-响应”三个核心维度描述，将模糊的质量要求转化为可衡量的具体场景：

1. **刺激：**触发场景的事件来源，比如“并发用户数涨到1万”“需要新增一个业务模块”。
 2. **环境：**事件发生时系统的状态，比如“系统正常运行、负载60%”“开发周期紧张”。
 3. **响应：**系统受到刺激后的行为和结果，以及可衡量的指标，比如“系统响应时间小于2秒”“新增功能开发周期不超过2周”。
-

第6题

软件体系结构设计的主要目的是满足对软件的_____。

答案：质量属性需求（非功能需求）

详细讲解：

软件需求分为两类：功能需求（系统“能做什么”）和非功能需求（系统“做得怎么样”，即质量属性）。功能需求决定了系统的功能模块划分，而**体系结构设计的核心目标，是满足软件的质量属性需求**——比如高并发需求对应分布式架构，高可靠需求对应冗余备份架构，易维护需求对应分层解耦架构。可以说，功能是架构的基础，而质量属性是架构设计的核心关注点和主要目的。

第7题

软件重用是为了解决_____。

答案：软件危机

详细讲解：

软件危机的核心痛点是：重复开发多、开发效率低、成本高、质量不可控。软件重用（软件复用）正是为了解决这些问题：在多次软件开发中，重复使用成熟的软件元素（代码、构件、设计、架构、文档等），不用每次从零开发。

软件重用可以大幅减少重复劳动，提升开发效率，降低成本；同时重用经过验证的成熟构件，也能提升软件质量和可靠性，是缓解软件危机的核心手段之一。

扩展知识点：软件重用的粒度从低到高包括：代码级重用、构件级重用、设计模式重用、架构风格重用、需求/文档级重用。

第8题

在多智能体系统中，负责接收用户请求并将任务分发给不同智能体的组件称为_____。

答案：路由（编排器/协调智能体）

详细讲解：

多智能体系统（Multi-Agent System）由多个具备不同专长的智能体组成，通常采用层级协作结构：

- **协调/主管智能体（Supervisor Agent）**：相当于“管理者”，负责接收用户的整体请求，拆解任务并分发给对应的专业智能体，最后汇总结果返回给用户，承担调度、协调的职责。
 - **工作智能体**：相当于“执行者”，各自负责特定领域的任务，比如代码编写、数据分析、文案创作。这种层级结构可以通过分工协作，高效处理复杂的多步骤任务。
-

第9题

软件体系结构应建立于传统软件开发过程的_____和_____阶段之间。

答案：需求分析、软件设计（总体设计/概要设计）

详细讲解：

传统瀑布模型的开发阶段顺序为：需求分析 → 总体设计（概要设计） → 详细设计 → 编码 → 测试 → 维护。

软件体系结构设计是承上启下的高层设计阶段：

- 承接**需求分析**的结果：基于需求明确系统的功能和质量目标。
 - 指导后续的**总体设计**：先确定系统的整体架构、风格、构件划分，再做具体的模块设计。它是从“系统要做什么”到“系统怎么做”的关键过渡。
-

第10题

C2 体系结构风格可以概括为通过_____绑定在一起的、按照一组规则运作的_____网络。

答案：连接件、并行构件

详细讲解：

C2架构风格是一种基于构件和连接件的分层架构风格，标准定义是：**通过连接件绑定在一起的、按照一组规则运作的并行构件网络。**

它的核心规则：

1. 系统中的构件和连接件都有顶部和底部。
2. 构件之间不能直接连接，必须通过连接件间接连接。
3. 构件之间通过消息异步通信，支持并发运行。

C2风格的优势是构件高度解耦、易扩展、支持并发，常应用于图形界面系统和分布式系统。

三、简答题

简答题与综合题的答案我以课上老师给出的简略答案和评分标准为主并结合多个AI的答案进行了调整，所以答案与AI讲解可能不太对应

第1题

软件体系结构的定义众多，你如何理解软件体系结构？它在软件系统中有何作用？

答案

理解：软件体系结构是软件系统的高层组织结构和设计方案，核心由**构件、连接件、约束**三部分组成。构件是承担功能的单元，分为处理构件与数据构件；连接件用于实现构件之间的交互与连接；约束规定了构件与连接件的组合规则与限制。它相当于建筑的“结构图纸”。

作用：

1. **沟通媒介：**为不同利益相关者（客户、开发人员、项目经理）提供统一的视图。
2. **早期决策：**对系统的质量属性（性能、安全、可维护性等）做出关键约束。
3. **可重用性：**体系结构级的模式和风格可在不同项目中复用，提高开发效率。
4. **项目管理基础：**便于任务分解、预算估算和进度规划。
5. **演化支撑：**良好的结构使系统易于修改和扩展，延长软件生命周期。

详细讲解

我们可以把软件体系结构类比为**盖房子的总设计蓝图**：盖房子前要先画图纸，规定哪里是客厅、哪里是卧室、水电怎么走、承重墙不能拆；软件体系结构就是软件的“整体设计图纸”，在写代码前就定好系统的骨架。

核心三要素解释

- **构件：**相当于房子里的客厅、卧室、厨房，每个单元有独立功能。软件中的构件就是一个个功能模块，比如登录模块、支付模块；有的负责加工数据（处理构件），有的负责保存数据（数据构件）。
- **连接件：**相当于房子里的电线、水管、走廊，负责连接各个房间，实现资源流通。软件中的连接件就是模块之间的通信方式，比如函数调用、消息队列、网络请求，让不同构件之间传递数据、协同工作。

- **约束**：相当于房子里“承重墙不能拆”“水电必须符合安全规范”的规矩。软件中的约束就是构件连接时必须遵守的规则，比如只能上层调用下层、数据只能单向流动，保证系统结构稳定。

作用的通俗解释

1. **沟通媒介**：一个项目有产品、开发、测试、运维很多角色，大家不用都去读代码，看架构图就能明白系统整体结构，方便跨角色沟通。
2. **早期决策**：写代码之前就定好架构，相当于先定好整体方向，后续的开发分工、模块拆分都跟着架构走，避免写到一半发现结构全错了。
3. **架构复用**：成熟的架构（比如三层B/S架构）可以直接套用到多个不同业务的系统里，不用每次从零开始设计，就像商品房的户型图可以反复使用。
4. **决定质量**：架构选得好不好直接决定系统好不好用。比如用了分层架构，改界面就不用动数据库，可维护性就好；用了管道过滤器，处理大批量数据的性能就高。
5. **预测质量**：不用等代码全部写完，在架构设计阶段就能通过分析评估，大致判断系统性能够不够、安不安全，提前规避风险。
6. **方便迭代**：软件需求经常变更，好的架构改一处不会牵连其他部分。比如换数据库只需要改数据层，不用改前端界面，改动成本很低。
7. **支撑原型与培训**：可以基于架构快速做简易原型验证想法；新员工入职先学系统架构，就能快速理解整体逻辑，不用逐行读代码。

第2题

试分析软件体系结构测试和传统软件测试区别。

答案

1. **测试对象不同**：传统软件测试针对程序代码、功能逻辑；体系结构测试针对架构设计方案，不关注具体实现代码，检查设计的合理性与一致性。
2. **测试阶段不同**：传统测试在代码开发完成后开展；体系结构测试是在软件设计阶段之前。
3. **测试重点不同**：传统测试关注功能是否正确、是否存在代码bug；体系结构测试关注性能、可靠性、可扩展性、可维护性等属性。
4. **测试目的不同**：传统测试验证代码实现的正确性、功能是否满足要求；体系结构测试验证高层结构设计是否满足质量属性需求。
5. **测试依据不同**：传统测试依据需求规格说明、代码逻辑；体系结构测试依据架构描述文档、风格约束、接口规约。

详细讲解

先明确两个概念的本质：

- **传统软件测试**：就是常说的“找bug”，写完代码后跑测试用例，验证功能对不对、会不会崩溃，比如登录能不能成功、支付金额计算是否正确，完全围绕写好的代码展开。
- **软件体系结构测试**：是“对着设计图纸检查”，还没写代码的时候，就校验架构方案本身合不合理，比如两个模块能不能对接、数据能不能正常流转、会不会出现系统卡死。

逐条区别详解

1. 测试对象

传统测试的对象是“成品”——写好的程序、代码、功能；

架构测试的对象是“设计图”——架构方案、构件设计、连接件规则，完全不涉及具体代码实现。

2. 测试时机

传统测试是“事后检查”，代码开发完成后才进行，属于开发后期环节；

架构测试是“事前把关”，设计阶段就完成校验，在写代码之前就修正架构缺陷。如果等代码写完才发现架构错了，返工成本会非常高。

3. 测试内容

传统测试查“功能对不对”，比如输入正确密码能否登录、输入错误会不会提示，都是具体的功能点；

架构测试查“结构通不通”，比如：构件的接口是否匹配（类似插头和插座型号是否对应）、连接件的约束有没有遵守（比如数据是否单向流动）、系统内的消息能否从起点传到终点、并发场景下会不会出现死锁。

4. 测试目的

传统测试的目标是找出代码里的bug，保证程序运行不出错；

架构测试的目标是找出设计上的缺陷，保证系统各个部分能组装到一起协同工作，并且后续好修改、好扩展。

5. 测试依据

传统测试照着需求文档、产品说明来测，验证实现和需求是否一致；

架构测试照着架构设计文档、接口规范、架构风格规则来测，验证设计是否符合约定的约束。

可以用盖房子类比：传统测试相当于房子盖好后，检查墙直不直、水龙头出不出水；架构测试相当于施工前，检查设计图纸里的水电管线能不能接上、承重墙设计合不合理、消防通道通不通。

第3题

什么是软件体系结构风格？

答案

软件体系结构风格是描述某一特定应用领域中系统组织方式的惯用模式，反映了领域中多个系统共有的结构和利益特征。

它定义了一套设计词汇表（如构件类型和连接件类型），规定了构件与连接件的约束。

详细讲解

可以用建筑风格类比：软件体系结构风格就像中式风格、欧式风格、现代简约风格等建筑风格，每种都有固定的设计特点和成熟套路，不用每次从零开始设计房子。

核心内涵拆解

1. 是经过验证的惯用模式

架构风格不是凭空设计的，是无数项目实践下来，被证明好用、靠谱的设计套路，被广泛复用后形成了固定的风格。比如做网站常用三层B/S风格，做数据处理常用管道-过滤器风格。

2. 有专属的设计词汇表

每种风格都有自己专属的构件和连接件名称。比如管道-过滤器风格里，构件叫“过滤器”，连接件叫“管道”；C/S风格里，构件是客户端和服务端，连接件是网络请求。

3. 有明确的约束规则

每种风格都有必须遵守的设计规矩，保证风格的特性。比如管道-过滤器风格要求数据只能单向流动；分层架构要求只能上层调用下层，不能越级调用。

使用架构风格的价值

- **降低设计成本**：成熟的风格可以直接复用，不用从零摸索系统结构，比如做信息管理系统直接用分层架构，节省设计时间。
- **提升可理解性**：开发人员只要知道系统用了某种风格，立刻就能明白系统的大致结构，不用从头读完整份设计文档。
- **统一设计规范**：风格本身已经解决了很多共性问题，比如分层架构解决了模块耦合的问题，使用后不用再重复设计解耦方案。

常见的五大类经典架构风格包括：数据流风格、调用/返回风格、独立构件风格、虚拟机风格、数据为中心风格。

第4题

试分析三层 B/S 体系结构的优缺点。

答案

三层B/S体系结构由浏览器（表示层）、Web服务器（业务逻辑层）、数据库服务器（数据层）三层组成。

优点：

1. 客户端零维护（只需浏览器）。
2. 系统升级维护仅需修改服务器端，运维成本低。
3. 跨平台性和扩展性较好。

缺点：

1. 交互性和实时性不如 CS 系统，安全性和并发能力相对依赖于服务器。

详细讲解

先明确三层B/S的基本结构：

B/S是Browser/Server（浏览器/服务器）的缩写，三层分别对应：

- **表示层**：用户电脑上的通用浏览器（如Chrome、Edge），只负责显示页面、接收用户输入，不需要安装专门的客户端软件。
- **功能层**：Web应用服务器，负责处理业务逻辑，比如登录校验、订单计算、规则判断。
- **数据层**：数据库服务器，负责数据的存储、读写与管理。

我们日常使用的网页版教务系统、电商网站、在线办公系统，都属于三层B/S架构。

优点详解

1. 零客户端部署

用户不用下载安装包，只要有浏览器、能上网就能使用系统，推广门槛极低。比如用户不用装“百度客户端”，打开浏览器就能搜索，对终端几乎没有要求。

2. 维护升级便捷

如果要修改功能、更新版本，只需要修改服务器上的代码即可，用户完全无感知，刷新页面就能使用新版本。不像传统客户端软件，每次升级都要用户手动下载安装包，逐台更新成本很高。

3. 支持跨地域访问

只要有网络，任何地点都能访问系统，非常适合互联网产品、跨地区的企业系统，满足分布式办公的需求。

缺点详解

1. 浏览器兼容问题

不同厂商的浏览器对页面代码的解析规则不同，同一个页面在Chrome、Firefox、旧版IE中可能显示错位、功能失效，开发时需要做大量适配工作，增加了开发成本。

2. 性能弱于C/S架构

因为核心运算都在服务器上完成，浏览器只负责展示，数据还要通过网络传输，响应速度和处理能力都不如安装在本地的客户端软件。比如大型3D游戏、专业设计软件都采用C/S客户端架构，就是因为B/S性能无法满足需求。

3. 安全与速度成本高

数据通过公网传输，容易被窃取或篡改，需要做加密、防护、权限控制等安全设计，成本很高；要保证多用户同时访问不卡顿，需要采购高性能服务器、优化带宽，硬件投入远大于C/S架构。

补充对比：两层C/S（客户端/服务器）需要安装专用客户端，优点是性能好、功能强，缺点是升级麻烦、部署成本高；B/S是C/S的演进，用通用浏览器替代了专用客户端，牺牲了部分性能，换来了部署维护的便捷性。

第5题

试分析管道-过滤器风格的结构特点

答案

由过滤器（构件）和管道（连接件）组成，过滤器负责数据处理，管道负责数据传输。

优点：

1. 过滤器的职责单一，相互独立，具有低耦合性。
2. 过滤器可以串联并联，复用性和扩展性较好。

缺点：

1. 数据通常是单向流动的，不适合较强的交互业务。
2. 频繁的数据转换可能给管道传输带来额外的开销。
3. 过滤器是无状态的，不适合处理上下文。

详细讲解

可以用**工厂流水线**类比管道-过滤器风格：原材料（数据）从一端进入，经过第一个工位（过滤器1）加工，通过传送带（管道）送到第二个工位（过滤器2）继续加工，最终产出成品。每个工位只负责自己的工序，不用关心其他工位的工作方式，传送带只负责单向传输物料。

特点逐条详解

1. 构件是过滤器

过滤器是数据处理单元，输入是数据流，输出也是数据流。比如一个过滤器负责“清洗无效数据”，下一个负责“统计数据指标”，再下一个负责“生成报表”。

过滤器是完全独立的：它不保存全局状态，也不知道上游是谁、下游是谁，只负责把输入的数据处理完输出。

2. 连接件是管道

管道是连接两个过滤器的“数据通道”，数据只能单向向前流动，不能反向回传。管道通常带有缓冲能力，类似传送带可以暂存物料，当前端处理快、后端处理慢时，数据可以先存在管道里，不会立刻导致系统阻塞。

3. 数据驱动执行

和普通按代码顺序执行的程序不同，管道-过滤器系统中，只有当数据送达过滤器的输入端口时，过滤器才会启动处理；没有数据时过滤器处于空闲状态。整个系统的运行完全由数据流动推动，而不是由固定的代码执行顺序控制。

4. 松耦合、易复用

因为过滤器之间只通过管道传递数据，互相不依赖，所以只要输入输出的数据格式匹配，过滤器可以随意替换、拆卸、重组。比如把“统计数据”的过滤器换成新算法版本，其他部分完全不用修改；也可以把过滤器拆下来，用到其他数据处理流水线中，复用性极强。

5. 天然支持并行

各个过滤器独立运行，第一个过滤器处理完第一段数据传给第二个，就可以立刻处理第二段数据，第二个过滤器同时处理第一段数据，就像流水线一样并行工作。因此管道-过滤器处理大批量流式数据的效率很高，适合视频编码、日志分析、数据ETL等场景。

第6题

请简述 MVC，并介绍各模块的作用和用途。

答案

MVC是一种将数据展示、业务逻辑与用户交互分离的设计模式。

- 模型 (Model)**：负责封装系统的核心数据与业务规则，是系统的功能核心；不依赖视图与控制器，独立存储状态、处理业务逻辑。
- 视图 (View)**：负责数据的可视化展示，向用户呈现界面；不处理业务逻辑，仅基于模型的数据渲染界面。
- 控制器 (Controller)**：接收用户输入，调用 Model 进行处理，并选择合适 View 返回结果，协调模型与视图的交互。

MVC实现了展示与逻辑的解耦，提升了代码的可维护性、可扩展性与并行开发效率。

详细讲解

用餐厅分工类比MVC：

- 模型 (Model) = 后厨：负责做菜（处理业务逻辑、管理数据），掌握所有菜品配方和食材，不管客人长什么样、怎么点菜，收到订单就按规则做菜。
- 视图 (View) = 餐桌和菜单：负责给客人展示信息（呈现界面），客人通过菜单点菜，它只负责展示，不参与做菜。

- 控制器 (Controller) = 服务员：客人点菜 (用户操作)，服务员把订单传给后厨 (调用模型)；菜做好了，服务员端到桌上给客人 (更新视图)。客人和后厨不直接接触，全靠服务员中转。

各模块作用详解

1. 模型 (Model)

是系统的“数据核心+业务大脑”，存储所有核心数据，也封装了所有业务规则。比如电商系统中，商品数据、订单数据、“下单扣库存”的规则，都定义在模型中。

它是最独立的部分，完全不依赖界面，核心逻辑不会因为视图变化而改动，因此复用性极强——同一个模型可以对应多个不同的视图，比如商品数据既可以在列表页展示，也可以在详情页展示。

2. 视图 (View)

是系统的“交互界面”，就是用户看到的页面/窗口。它只负责一件事：把模型里的数据展示给用户，接收用户的输入操作。

它不处理任何业务逻辑，比如界面上显示商品价格，它只负责把数字渲染出来，不管价格是怎么计算出来的；用户点击按钮，它也不自己处理，直接转交给控制器。

3. 控制器 (Controller)

是中间的“调度员+传话筒”。用户点击“下单”按钮，视图先收到操作，然后告诉控制器“用户触发了下单操作”；控制器就去调用模型里的“创建订单”方法；模型处理完，告诉控制器“订单创建成功”；控制器再通知视图“更新界面，显示订单成功状态”。

它负责把用户的操作翻译成模型能识别的指令，再把模型的结果翻译成视图能展示的内容，是两者之间的桥梁。

MVC的核心价值

- **解耦**：界面和逻辑彻底分开，改界面不用动业务代码，改业务不用动界面，比如把网页端换成手机端，模型完全不用重写。
- **并行开发**：前端工程师写视图，后端工程师写模型，控制器负责对接，团队可以并行工作，提升开发效率。
- **易维护**：出问题定位清晰，界面显示错误找视图，业务计算错误找模型，交互逻辑错误找控制器，分工明确。

补充：MVC最早是为图形界面 (GUI) 设计的，现在Web开发、移动端开发中广泛使用，还衍生出了MVP、MVVM等变体，但核心思想都是“展示与逻辑分离”。

第7题

请描述模型上下文协议 (MCP) 的实现原理，及其与 A2A 协议的区别。

答案

1. MCP协议通过三层C/S架构实现。
2. 客户端由 AI Agent 或上层应用承担，对接模型并统一发起资源调用请求。
3. 服务端封装数据库、API、文件系统等外部资源，对外输出标准化能力。
4. 通过统一协议规范，实现模型对外部工具的自动发现、调用与返回结果，无需逐个工具单独适配。
5. **区别**：MCP与A2A都采用三层C/S架构实现，但MCP 是智能体与外部工具的接入协议，A2A 是多智能体间的协作通信协议。

详细讲解

先铺垫背景：大语言模型本身只能生成文字，要实现查天气、读文件、控制系统等能力，就需要调用外部工具。早期每个大模型厂商都有自己的工具调用规范，互不兼容，一个工具只能适配一个模型，复用成本很高。MCP就是为了解决这个问题的统一行业标准。

MCP原理解读

可以把MCP理解成**通用插座标准**：以前每个电器（工具）都有专属插头，只能插对应的插座（特定大模型）；有了MCP统一标准后，所有工具都做成MCP规格，所有支持MCP的大模型都能直接调用，实现了“一次开发，处处可用”。

- MCP客户端：在大模型/智能体一侧，相当于“插头”，负责发现工具、发送调用请求、接收返回结果。
- MCP服务器：在工具/数据源一侧，相当于“插座”，负责声明自身能力、接收请求、调用真正的工具执行、返回结果。

工作流程类似住酒店呼叫服务：

1. 你打电话问前台“你们有哪些服务？”，前台告诉你有打扫、送餐、洗衣（对应发现阶段：客户端查询服务器能力）。
2. 你说“我要一份炒饭”（对应请求阶段：客户端发起工具调用，指定功能和参数）。
3. 前台通知后厨制作炒饭（对应执行阶段：服务器调用底层服务）。
4. 服务员把炒饭送到房间，告知你“餐已送达”（对应响应阶段：服务器返回标准化结果，客户端同步给大模型）。

A2A与MCP的区别解读

A2A是智能体和智能体之间通信的协议，相当于**不同公司员工协作的通用工作语言**。比如一个智能体擅长写代码，另一个擅长做测试，二者可以通过A2A协议分工合作，共同完成项目。

逐条对比：

1. 目标不同

MCP是“人用工具”的协议：大模型（人）通过MCP调用外部工具（计算器、数据库）。

A2A是“人和人合作”的协议：一个智能体（人）通过A2A和另一个智能体（人）协同完成任务。

2. 地位不同

MCP是主从关系：客户端（大模型）主动发起调用，服务器（工具）被动执行，工具没有自主决策能力。

A2A是对等关系：双方都是具备自主能力的智能体，可以主动发起请求，也可以响应请求，比如A委托B干活，B也可以反过来向A索要信息。

3. 范围不同

MCP只负责“工具调用”这一件事，比如调用计算器算数值、调用接口查数据。

A2A覆盖“智能体协作”的全流程，比如下发任务、传递文件、同步进度、汇报结果，功能比MCP复杂得多。

4. 交互方式不同

MCP基本是“一问一答”的同步模式，调用一次、返回一次结果，类似使用计算器。

A2A支持多种模式：立刻返回结果（同步）、先接收任务后续反馈（异步轮询）、边执行边发进度（流式更新）、完成后主动通知（Webhook），适合复杂的长期任务。

第8题

请写出管道-过滤器风格体系结构的可靠性建模公式。

答案

$$M(i, j) = \begin{cases} R_i \cdot P_{ij}, & S_i \notin S_p \\ \prod_{C_n \in S_i} R_n \cdot P_{nj}, & S_i \in S_p, \quad 1 \leq i, j \leq |S|, \quad 1 \leq n \leq k \\ 0, & S_i \text{ 无法到达 } S_j \end{cases}$$

其中：

- $M(i, j)$ ：状态转移矩阵的元素。 i 是起始状态的编号， j 是目标状态的编号。
- S_i / S_j ：系统的第 i / 第 j 个状态； $|S|$ 表示系统所有状态的总数量。
- R_i ：状态 S_i 自身的可靠度，即系统处于 S_i 状态时、该状态下正常工作的概率。
- P_{ij} ：状态转移概率，即系统从状态 S_i 正常流转到状态 S_j 的概率。
- C_n ：系统中的第 n 个构件。
- S_p ：并行状态集合，即所有中间并行构件共同构成的一个合并状态。
- 起始状态不在并行段 ($S_i \notin S_p$)：属于普通串行管道转移，可靠度 = 当前状态可靠度 × 转移概率，即 $R_i P_{ij}$ 。
- 起始状态在并行段 ($S_i \in S_p$)：并行段所有构件都要成功转移，因此对所有并行构件的 $R_n \cdot P_{nj}$ 做连乘。
- 两个状态无转移路径：转移概率记为0。

详细讲解

可靠性的本质是“系统在指定时间、指定条件下正常运行的概率”。管道-过滤器是流式结构，数据沿着管道依次经过各个过滤器，因此适合用状态迁移模型来描述数据流动的过程，计算系统正常运行的概率。

通用建模方法解读

1. 第一步：拆分状态，建迁移矩阵

把数据处理的每个阶段拆成一个状态：数据进入过滤器1是状态1，处理完进入过滤器2是状态2.....最终处理完成是状态k。

每个状态自己有可靠性(R_i)，就是对应过滤器正常工作的概率；两个状态之间有迁移概率(P_{ij})，就是数据从i流到j的概率。

矩阵里的每个值($M(i,j)$)，就是“状态i正常工作，并且数据成功流到状态j”的概率，等于自身可靠性乘以迁移概率。

2. 第二步：计算累积矩阵T

因为数据可能在状态间多次流转（比如出错重试），不止走一步，所以要把走1步、2步.....无数步的概率全部加起来。数学上这个无穷等比矩阵级数可以简化为 $(I-M)^{-1}$ （单位矩阵减M的逆矩阵），这个推导叫诺伊曼级数展开，和等比数列求和原理一致。

3. 第三步：算系统总可靠性

矩阵T里第1行第k列的数，就是从初始状态走到最终成功状态的总概率；再乘以最终状态自身的可靠性，就是整个系统的可靠性。

简化串联公式解读

如果是最标准的管道-过滤器，数据只往前流、没有分支、没有回流，就是纯串联结构，像一串串联的灯泡，一个坏了整条路就断了。这时候不用复杂的矩阵计算，直接把所有过滤器的可靠性相乘即可。

举个例子：3个过滤器，可靠性分别是0.9、0.8、0.7，系统可靠性就是 $(0.9 \times 0.8 \times 0.7 = 0.504)$ ，即50.4%的概率正常运行。

这个简化公式是通用状态迁移模型的特例，因为纯串联时每个状态只会迁移到下一个状态， $(P_{\{i(i+1)\}}=1)$ ，其他迁移概率为0，最终推导结果就是乘积。

第9题

请写出容错风格体系结构的可靠性建模公式。

答案

$$M(i, j) = \begin{cases} R_i \cdot P_{ij}, & S_i \notin S_b \\ R_{a_1} + \sum_{q=a_2}^{a_r} \left(\left(\prod_{m=1}^{q-1} (1 - R_m) \right) \cdot R_q \right), & S_i \in S_b \text{ 且 } S_i \text{ 包含构件 } C_{a_1} \cdots C_{a_r} \\ 0, & S_i \text{ 无法到达 } S_j, \quad 1 \leq i, j \leq |S|, 1 \leq a_r \leq k \end{cases}$$

其中：

- $M(i, j)$ ：状态转移矩阵的元素。 i 是起始状态的编号， j 是目标状态的编号。
- S_i / S_j ：系统的第 i / 第 j 个状态； $|S|$ 表示系统所有状态的总数量。
- R_i ：状态 S_i 自身的可靠度，即系统处于 S_i 状态时、该状态下正常工作的概率。
- P_{ij} ：状态转移概率，即系统从状态 S_i 正常流转到状态 S_j 的概率。
- S_b ：关键状态集合，也就是需要部署冗余、实现容错的状态集合。
- $C_{a_1}, C_{a_2}, \dots, C_{a_r}$ ：容错段的 r 个冗余构件。
- a_r 表示第 r 个冗余构件的编号，满足 $a_r \leq k$ （不超过系统总构件数）， a_1 是第一个主用构件， $a_2 \sim a_r$ 是后续的备用冗余构件。
- q ：求和的循环下标，从第2个冗余构件 a_2 遍历到最后一个 a_r ，代表“第 q 个构件首次成功”的场景。
- m ：连乘的循环下标，遍历前 $q - 1$ 个构件，代表“前 $q - 1$ 个构件全部失败”的场景。
- $1 - R_m$ ：第 m 个构件的失效概率。
- 容错的本质是“至少一个构件成功就算成功”，因此把所有“首次成功”的场景概率相加：
 - R_{a_1} ：第一个主构件直接成功的概率；
 - $\left(\prod_{m=1}^{q-1} (1 - R_m) \right) \cdot R_q$ ：前 $q - 1$ 个构件全部失效，第 q 个构件成功的概率；
 - 对所有 q 求和，就是整个容错段“至少一个成功”的总可靠度。

详细讲解

容错的核心思想是**冗余备份**：就像飞机有多个发动机，坏一个还有其他的能工作；服务器有主备两台，主的挂了备用的立刻顶上，系统不会中断。容错风格就是用多个冗余组件，提升系统整体的可靠性。

单状态冗余可靠性解读

一个关键节点配置了多个冗余组件，只要有一个组件正常，这个状态就能正常工作。
成功的场景有多种：

- 主组件正常工作，不用启动备用：概率(R_{a1})
- 主组件坏了，第一个备用组件顶上：概率($(1-R_{a1}) \times R_{a2}$)
- 主和第一个备用都坏了，第二个备用顶上：概率($(1-R_{a1}) \times (1-R_{a2}) \times R_{a3}$)
-以此类推，把所有“第一个正常工作的备份”的概率加起来，就是这个状态总的有效可靠性。

举个例子：主组件可靠性0.9，第一个备用0.8，第二个备用0.7

状态可靠性 = ($0.9 + (1-0.9) \times 0.8 + (1-0.9) \times (1-0.8) \times 0.7 = 0.9 + 0.08 + 0.014 = 0.994$)
也就是99.4%的概率正常工作，远高于单个组件的0.9，这就是冗余的价值。

系统整体可靠性解读

算出每个状态的有效可靠性之后，后续步骤和管道-过滤器的建模完全一致：构建迁移矩阵、计算累积矩阵、得到系统总可靠性。区别只在于单个状态的可靠性计算：管道-过滤器是单组件可靠性，容错风格是多冗余组件的综合可靠性。

纯并联简化公式解读

如果系统就是简单的并联结构，多个组件干同一件事，只要有一个好的系统就正常，那系统正常的概率 = $1 - \text{所有组件全都坏掉的概率}$ 。

举个例子：两个组件并联，可靠性都是0.9

都坏的概率是($0.1 \times 0.1 = 0.01$)，系统可靠性就是($1 - 0.01 = 0.99$)。
这是容错最基础的公式，也叫并联系统可靠性公式，是通用建模方法的特例。

第10题

请描述检索增强生成（RAG）的原理与实现过程。

答案

原理

它从外部知识库中检索与用户问题相关的文档片段，将其作为上下文补充进大模型的提示词中，引导模型生成更准确、更具时效性的答案。

实现过程

1. **建库**：将原始文档分块，通过 Embedding 模型转换成向量，存入向量数据库。
2. **检索**：用户提问后，将问题也转为向量，在库中检索相似度最高的 top-k 个文档块。
3. **提示增强**：将检索到的文档块与问题拼接成 Prompt，输入大语言模型。
4. **生成**：模型基于提供的上下文生成最终答案。

详细讲解

先讲RAG出现的背景：大语言模型的知识都来自训练数据，天生有三个缺陷：

1. **知识过时**：模型训练完成后，训练数据就固定了，无法知道训练之后发生的新事件。
2. **幻觉问题**：模型会编造看似合理但完全错误的答案，可信度不足。

3. **缺乏私有知识**：企业内部文档、项目规章等私有数据，模型训练时接触不到，无法回答相关问题。RAG就是专门解决这些问题的技术，核心思想就是**让大模型“开卷考试”**：回答问题前先查资料，再照着资料回答。

原理解读

RAG就像开卷考试：闭卷考试时，学生全靠脑子里记的知识答题，容易记错、不会新题；开卷考试允许翻参考书，找到和题目相关的内容，再照着书写答案，既准确又能覆盖新知识，还不会瞎编。

RAG里的“参考书”就是外部知识库，“翻书”就是检索过程，“答题”就是大模型生成过程。

实现过程详解

离线阶段：搭建知识库（相当于整理书本、做索引）

1. **文档采集**：把所有要用到的资料收集起来，比如公司员工手册、产品文档、历史公告、业务规则，都是原始的文字材料。
2. **文档分块**：一本书太厚，不能整本塞给大模型，也不好检索。所以把长文档拆成一小段一小段的文本块（比如每段500字），拆分时保证语义完整，既方便检索，也能塞进模型的输入窗口。
3. **向量嵌入**：文字没法直接比“语义相似度”，所以用嵌入模型把每一段文字转换成一串数字（向量），语义越相近的文字，对应的向量距离越近。比如“猫咪”和“小猫”的向量距离很近，和“汽车”的距离很远。
4. **向量存储**：把所有文本块的向量，连同原文一起，存入专门的向量数据库。向量数据库擅长快速查找“和目标向量最相似的向量”，相当于图书馆的检索索引，能快速找到和问题相关的“书页”。

在线阶段：用户提问（相当于翻书答题）

1. **查询向量化**：用户提出问题后，先用同一个嵌入模型，把问题也转换成向量，这样才能和库里的向量比相似度。
2. **语义检索**：拿问题向量去向量数据库里匹配，找出最相关的前N个文本块，也就是和问题最沾边的几段资料。它是按语义匹配，不是简单的关键词匹配，比如用户问“怎么请假”，哪怕资料里写的是“休假申请流程”，也能被检索到。
3. **提示增强**：把检索到的相关文本，和用户的问题拼在一起，组成完整的提示词发给大模型，格式类似“请根据下面的参考资料回答用户问题：参考资料：xxx 用户问题：xxx”。
4. **生成回答**：大模型照着参考资料生成回答，这样答案有事实依据，不会凭空编造；而且知识库可以随时更新，模型就能回答最新的知识。

补充：RAG不需要重新训练大模型，只要更新知识库就能更新知识，成本低、见效快，是企业搭建内部知识库、智能客服、问答系统最主流的技术方案。

四、综合题

第1题

某高校拟开发一个“在线考试系统”，系统主要功能包括：

- （1）学生登录系统后可以参加考试、提交试卷和查询成绩；
- （2）教师可以创建试卷、发布考试并批改试卷；
- （3）管理员负责用户管理和考试安排；
- （4）系统能够保存考试数据并支持成绩统计。

请设计该在线考试系统的软件体系结构，包括系统的主要模块、模块之间的关系以及采用的体系结构风格，并说明选择该体系结构风格的理由。

答案

一、采用的体系结构风格

采用**三层B/S体系结构风格**，划分为表示层、业务逻辑层、数据层；同时结合MVC模式拆分表示层与业务逻辑的交互，属于调用-返回风格的典型应用。

二、系统主要模块与模块关系

1. 表示层（浏览器端）

- 学生端界面：登录入口、考试答题页面、试卷提交、成绩查询页面
- 教师端界面：试卷管理、考试发布、试卷批改、成绩统计页面
- 管理员界面：用户管理、考试安排、系统配置页面

2. 业务逻辑层（Web服务器端）

- 用户管理模块：身份认证、权限校验、用户信息维护
- 考试管理模块：试卷创建、考试发布、时间控制、试卷提交校验
- 成绩管理模块：自动判分、人工批改、成绩计算、统计分析
- 系统管理模块：系统配置、操作日志、数据备份

3. 数据层（数据库端）

- 用户数据库：存储学生、教师、管理员的账号信息与权限数据
- 题库试卷数据库：存储题目、试卷配置、考试安排信息
- 成绩数据库：存储答卷、成绩、统计结果数据

模块间关系：表示层调用业务逻辑层，业务逻辑层调用数据层，自上而下单向依赖，下层不感知上层的存在。

三、选择该体系结构风格的理由

1. **部署推广便捷：**B/S架构无需安装客户端，师生通过浏览器即可访问；系统升级仅需修改服务器端，大幅降低运维成本。
2. **职责清晰易扩展：**三层结构实现了界面、业务、数据的解耦，便于团队并行开发和后期维护。例如修改考试规则仅需调整业务逻辑层，改动界面仅需调整表示层，互不影响，便于后续新增功能。
3. **数据安全性高：**业务逻辑层隔离了表示层与数据库，用户无法直接操作数据，保障考试数据的安全与完整性。
4. **技术成熟稳定：**三层B/S是Web系统的经典成熟方案，技术生态完善，开发成本低、可靠性高，适配在线考试对稳定性、并发能力的要求。

详细讲解

在线考试系统是典型的Web信息系统，用户分散、需要浏览器访问，因此三层B/S架构是最适配的选择。下面从零基础角度拆解设计思路：

为什么选三层B/S风格

高校的用户是学生和老师，分布在宿舍、机房、家里等不同地点，不可能要求每个人都安装专用客户端；同时系统需要经常更新题库、调整功能，服务器端统一升级最方便。三层B/S刚好满足这些需求，也是所有网页系统的通用标准架构。

模块设计思路

分层架构的核心是“各司其职、单向调用”，每层只干自己的事，只调用下一层，这样结构最清晰、改起来最方便。

- **表示层**就是大家看到的网页。不同角色登录后看到不同的菜单：学生看到考试、查成绩；老师看到出卷子、改卷子；管理员看到管账号、排考试。这一层只负责显示和收用户操作，不做任何业务计算，比如点“提交试卷”，它只是把请求发往后端，自己不算分。
- **业务逻辑层**是系统的“大脑”，所有规则都在这里。比如“考试时间到自动交卷”“客观题自动判分”“成绩按班级统计”，都写在这一层。它分成几个模块各管一块，收到前端请求后按规则处理，需要存数据、读数据时，就调用数据层。
- **数据访问层**就是管数据库的。所有用户信息、题目、试卷、分数都存在数据库里，这一层专门负责读写数据库，给业务层提供接口，业务层不用管数据库怎么存、存在哪，直接调用就行。

设计理由的场景化解读

1. 适配高校场景：用户量大、分布广，浏览器访问零门槛，不用安装软件，学生用手机、电脑都能登。
2. 维护成本低：比如要加一个“错题本”功能，只需要在业务层加逻辑、前端加页面、数据库加表，每层改动不影响其他部分。以后学校换数据库，只需要改数据访问层，业务和前端都不用动。
3. 保障考试公平：考试数据很敏感，不能让学生随便改。因为有业务层隔在中间，所有数据操作都要经过校验，比如是不是到考试时间了、有没有权限改分，用户碰不到底层数据库，安全性有保障。
4. 开发风险低：三层B/S是发展了几十年的成熟方案，各种技术框架都支持，开发时不容易踩架构层面的坑，系统稳定性有保障。

第2题

设计一个简单图书管理系统的软件体系结构，包括主要模块、模块关系、采用的体系结构风格，并说明理由。

答案

一、采用的体系结构风格

采用**三层C/S（客户端/服务器）层次体系结构**，属于调用-返回风格，分为表示层、业务逻辑层、数据层。

二、系统主要模块与模块关系

1. 表示层（客户端）

- 读者操作模块：图书检索、借阅记录查询、个人信息查看
- 管理员操作模块：图书入库、借阅登记、归还登记、读者管理、借阅统计

2. 业务逻辑层（应用服务器端）

- 借阅管理模块：借阅权限校验、还书日期计算、逾期规则处理、借阅与归还登记
- 图书管理模块：图书信息增删改查、分类管理、库存管理
- 读者管理模块：读者账号维护、借阅额度校验、信息更新
- 统计报表模块：借阅量统计、库存统计、逾期统计

3. 数据层（数据库服务器）

- 图书数据库：存储图书编号、书名、分类、库存、馆藏位置等信息

- 读者数据库：存储读者编号、姓名、权限、可借阅额度等信息
- 借阅记录数据库：存储所有借阅、归还、逾期记录

模块间关系：采用自上而下的单向调用，表示层依赖业务逻辑层，业务逻辑层依赖数据层，下层不依赖上层。

三、选择该体系结构风格的理由

1. **适配图书馆场景：**本地运算，响应速度快，大量图书借阅、批量导入卡顿少；可深度对接本地硬件：读卡器、条码枪、图书馆自助借还机；离线可用，断网时能临时办理借阅，联网后同步数据。
2. **职责清晰易扩展：**三层结构实现了界面、业务、数据的解耦，便于团队并行开发和后期维护。比如以后加“预约借书”功能，也只需要加对应的模块，不会打乱原有结构，便于后续新增功能。
3. **数据安全性高：**业务逻辑层隔离了表示层与数据库，用户无法直接操作数据，可有效避免图书数据、借阅记录被非法篡改或误操作。
4. **技术成熟稳定：**三层C/S是桌面业务系统的经典成熟方案，技术生态完善，开发成本低、可靠性高，适配图书管理对稳定性、本地硬件交互、离线操作的要求。

详细讲解

图书管理系统是典型的事务型信息管理系统，用户以馆内管理员为主，操作频繁、对响应速度有要求，因此三层C/S架构非常适配。

风格选择解读

图书管理系统大多部署在图书馆内部局域网，管理员每天高频使用，对操作流畅度要求高，C/S客户端的响应速度比网页更快；同时数据只在内部网络传输，安全性也更好。如果后续要做读者线上查书、预约借书，只需要把表示层换成网页，升级成B/S架构，业务层和数据层基本不用改，扩展性很好。

模块设计思路

- **表示层**就是操作软件的界面。管理员界面功能最全：新书入库、办理借书、办理还书、管理读者证、统计报表；读者界面功能简单：搜书、看自己的借阅记录、还书日期。如果是C/S架构，就是安装在电脑上的桌面软件。
- **业务逻辑层**管所有规则。比如“本科生最多借5本，借期30天”“逾期一天罚款0.1元”“图书借出后库存减1”，这些规则都写在这里。用户点“借书”，业务层先校验：读者有没有逾期未还的书？这本书还有没有库存？都符合条件才登记借阅、更新库存。
- **数据层**就是数据库，存所有图书信息、读者信息、借阅记录。业务层所有读写数据的操作都通过数据层，不用管底层数据库是MySQL还是Oracle。

设计理由解读

1. **结构清晰好开发：**分层之后分工明确，有人写界面、有人写业务逻辑、有人管数据库，团队可以并行开发，出了问题也容易定位。
 2. **方便后续修改：**比如图书馆调整规则，本科生最多借10本了，只需要改业务层里的借阅额度参数，界面和数据库都不用动。以后加“预约借书”功能，也只需要加对应的模块，不会打乱原有结构。
 3. **数据有保障：**图书和借阅数据是图书馆的核心数据，不能随便改。有业务层做校验，所有操作都要符合规则，不会因为误操作把库存改乱、把记录删了。
 4. **适配实际场景：**图书馆管理员每天都要用系统，局域网内的C/S客户端反应快、操作顺手，比网页体验好；如果以后要做线上服务，分层架构也能平滑升级，不会推倒重来。
-

第3题

某培训中心要研制一个培训管理系统软件，它的业务是：将学员发来的信件收集分类后，按几种不同的情况处理。

(1) 如果是报名的，则将报名数据送给负责报名事务的职员，他们将查阅课程文件，检查该课程是否额满，然后在学生文件、课程文件上登记，并开出报告单交财务部门，财务人员开发票给学生。

(2) 如果是想注销原来已选修课程，则由注销人员在课程文件、学生文件和帐目文件上做相应的修改，并给学生注销单。

(3) 如果是付款的，则由财务人员在帐目文件上登记，也给学生一张收费收据。

试根据要求画出该系统的数据流图，并转为软件结构图。

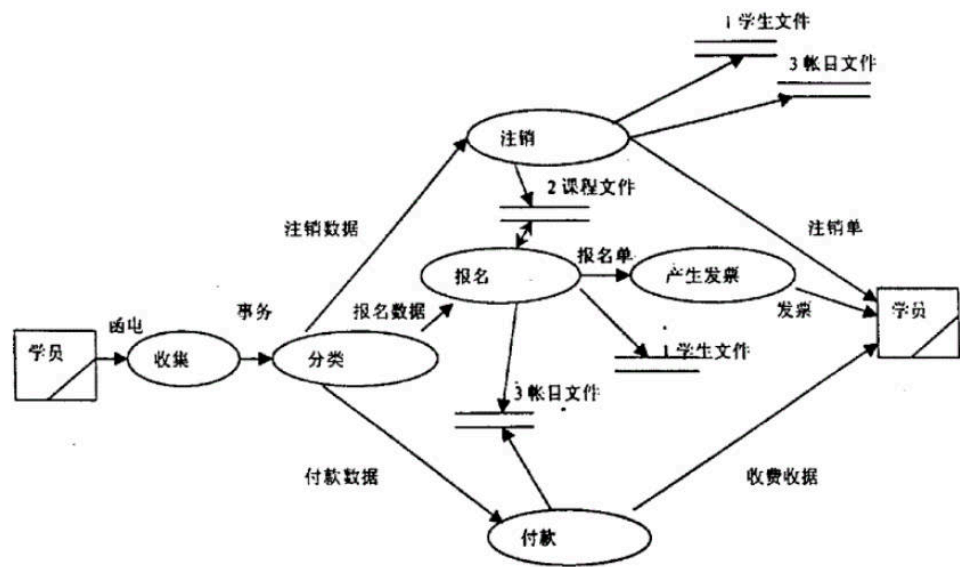
答案

软件工程应用题 - 百度文库

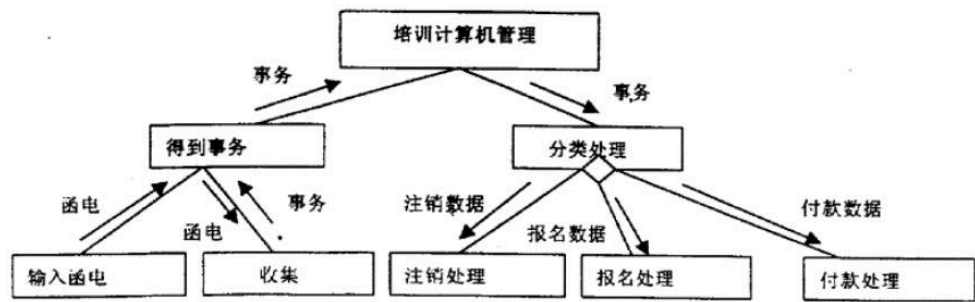
【数据流图到底怎么画？】 https://www.bilibili.com/video/BV1Zp421U7wh/?share_source=copy_web&vd_source=520d1b6bc322fb8a10e7a4602bad0f50

【软件结构图（仅提供一种解题思路）】 https://www.bilibili.com/video/BV1XH4y1H7ja/?share_source=copy_web&vd_source=520d1b6bc322fb8a10e7a4602bad0f50

(1)数据流图



(2)软件体系结构



培训计算机管理软件结构主图

详细讲解

先明确两个核心概念：

- **数据流图 (DFD)**：从“数据怎么流动”的视角描述系统，用圆圈表示加工（处理数据的步骤），方框表示外部实体（系统外的人/部门），双平行线表示数据存储（文件/数据库），箭头表示数据流。
- **软件结构图**：从“模块怎么调用”的视角描述系统，是树形的调用结构，上层模块调用下层模块，对应调用-返回风格的主程序-子程序模式。

数据流图设计思路

这个系统的业务逻辑是“输入分类 → 分情况处理 → 输出单据”，是典型的变换型数据流。

1. 顶层图把整个系统当成一个黑盒，只看输入输出：学员发信件进来，系统输出各种单据给学员和财务。
2. 1层图把黑盒拆开：第一步先分类，判断信件是报名、注销还是付款；然后分三个分支分别处理；每个处理流程读写对应的文件，最后输出结果。
 - 报名流程：查课程有没有名额 → 登记学生和课程文件 → 给财务发报告单开发票。
 - 注销流程：同时改学生、课程、帐目三个文件 → 给学员发注销单。
 - 付款流程：改帐目文件 → 给学员发收据。

数据流图转软件结构图的方法

这种变换型数据流，对应转换成“输入-处理-输出”三段式的软件结构，是结构化设计的标准方法（变换分析）：

- 输入部分对应“信件分类模块”，负责接收和分拣输入。
 - 中心处理部分对应“业务处理模块”，下面拆三个子模块对应三种业务。
 - 输出部分对应“单据输出模块”，统一负责生成和输出单据。
- 最上层放一个主控模块，统筹调度所有下层模块。

可以用公司组织架构类比：总经理（主控模块）管行政部（分类）、业务部（业务处理）、财务部（单据输出）；业务部下面再分报名组、注销组、付款组。上级调用下级，下级不反向调用上级，符合调用-返回的特点。

第4题

某电器集团公司下属的工厂包括技术科、生产科等基层单位。现在想建立一个计算机辅助企业管理系统，

其中生产科的任务是：

- (1) 根据销售公司转来的内部合同（产品型号、规格、数量、交货日期）制定车间月生产计划。
- (2) 根据车间实际生产日报表、周报表调整日生产计划。
- (3) 以月生产计划为依据，制定产品设计（结构、工艺）及产品组装月计划。
- (4) 将产品的组装计划传达给各科，将组装月计划分解为周计划，下达给车间。

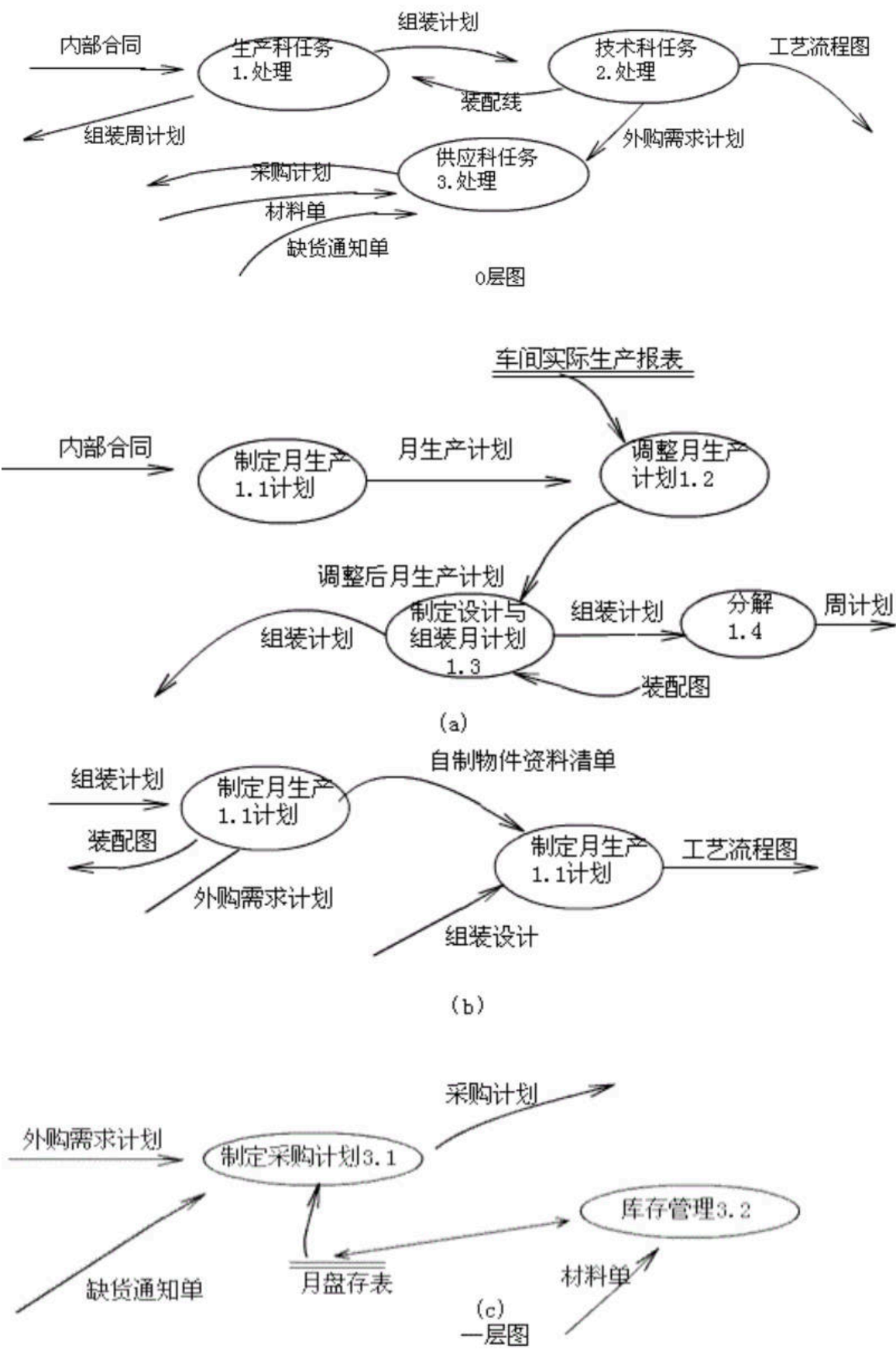
技术科的任务是：

- (1) 根据生产科转来的组装计划进行产品结构图设计，产生产品装配图给生产科，产生外购需求计划给供应科，并产生产品自制物料清单。
- (2) 根据组装计划进行产品工艺设计，根据产品自制物料清单产生工艺流程图给零件车间。

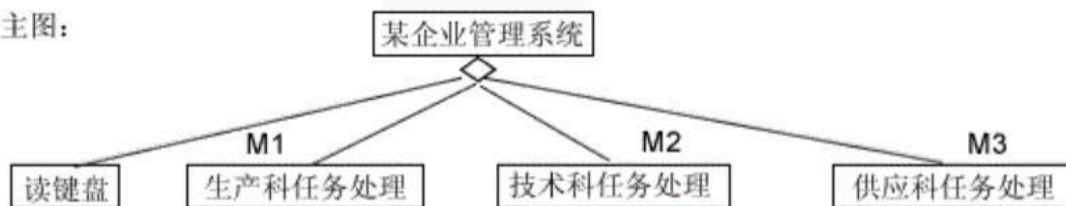
试根据要求画出该系统的数据流图，并转为软件结构图。

答案

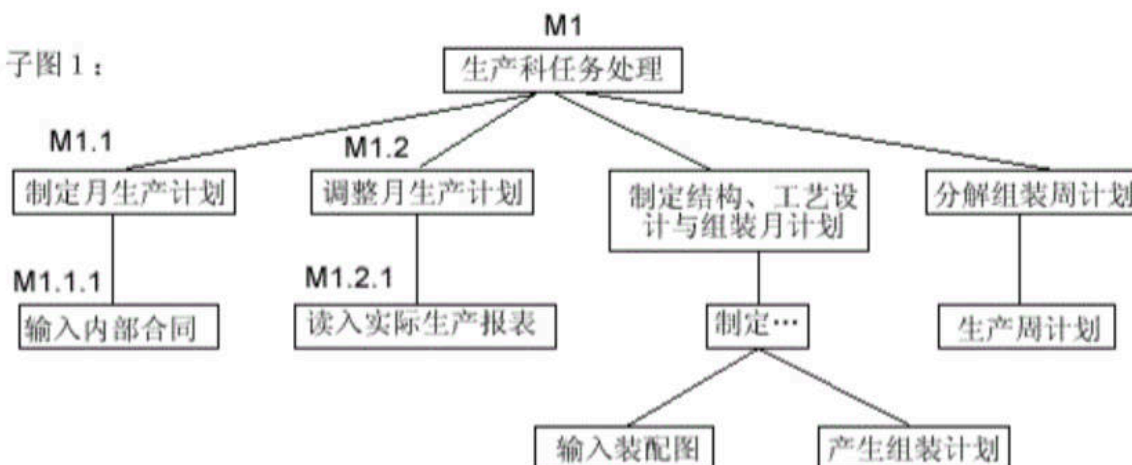
(第1章自测题及参考答案.doc



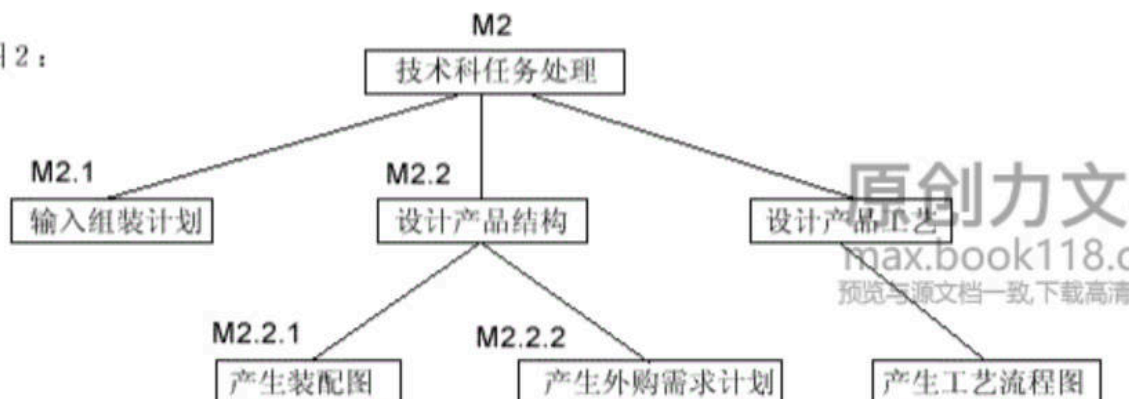
主图：



子图 1：



子图 2：



详细讲解

这个系统是典型的企业生产管理系统，业务按部门流转，数据从销售端流入，经过生产计划、技术设计，最终下发到执行部门。

数据流图设计思路

1. 顶层图把整个系统看作黑盒：输入是销售公司的内部合同、车间上报的生产报表；输出是给车间的计划、给供应科的外购需求、给零件车间的工艺图；系统内部存储计划、设计、工艺三类数据。
2. 1层图按部门拆分加工：
 - 生产科侧三个流程：先拿合同做月计划；再根据车间回报调整日计划；最后做组装计划，拆成周计划发下去，同时发给技术科。
 - 技术科侧两个流程：先做结构设计，出装配图、外购计划、物料清单；再做工艺设计，出工艺流程图给车间。所有数据都存在对应的数据存储里，各个加工通过读写数据存储传递信息。

转软件结构图的思路

按照部门和职能分层，对应企业的组织架构：

- 最顶层是系统总控，协调两个科室的流程。
- 第二层对应两个科室：生产科模块、技术科模块，分别管自己部门的业务。
- 第三层是每个科室下的具体功能模块，对应实际的业务岗位：生产科管计划、组装、下达；技术科管结构设计、工艺设计。

这种结构完全符合调用-返回的层次风格，上级模块调用下级模块，职责和企业部门分工一一对应，开发和落地都很顺畅；跨部门的数据通过共享数据库传递，不用模块之间直接调用，耦合度很低，后续部门职能调整也方便修改。